

蓝桥杯30天算法冲刺集训



Day-16 最小生成树与拓扑排序

生成树

对连通图进行遍历，过程中所经过的边和顶点的组合可看做是一棵普通树，通常称为生成树。

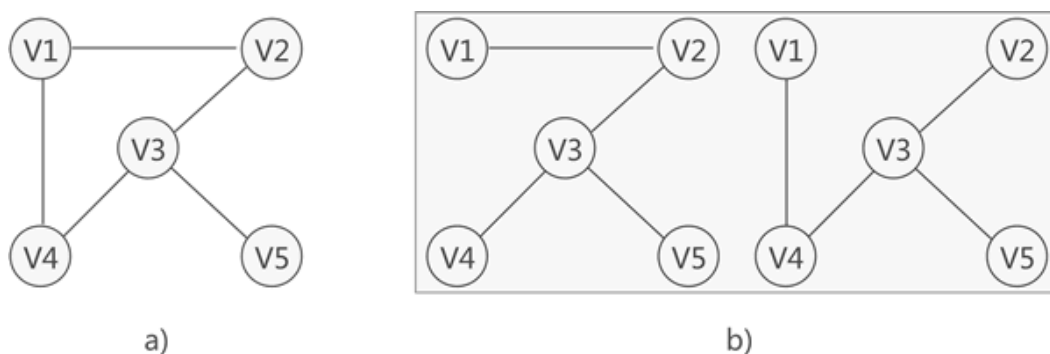


图 1 连通图及其对应的生成树

如图 1 所示，图 1a) 是一张连通图，图 1b) 是其对应的 2 种生成树。

连通图中，由于任意两顶点之间可能含有多条通路，遍历连通图的方式有多种，往往一张连通图可能有多种不同的生成树与之对应。

连通图中的生成树必须满足以下 2 个条件：

1. 包含连通图中所有的顶点；
2. 任意两顶点之间有且仅有一条通路；

因此，连通图的生成树具有这样的特征，即生成树中 边的数量 = 顶点数 - 1 。

最小生成树

我们定义无向连通图的 **最小生成树**（Minimum Spanning Tree，MST）为边权和最小的生成树。

注意：只有连通图才有生成树。

克鲁斯卡尔算法(Kruskal算法)

适用于稀疏图，时间复杂度 $O(m \log m)$ 。

核心思想：从小到大挑不多余的边，属于贪心的算法。

之前介绍了求最小生成树之普里姆算法。该算法从顶点的角度为出发点，时间复杂度为 $O(n^2)$ ，更适合与解决边的稠密度更高的连通网。

本节所介绍的克鲁斯卡尔算法，从边的角度求网的最小生成树，时间复杂度为 $O(E \log E)$ 。和普里姆算法恰恰相反，更适合于求边稀疏的网的最小生成树。

对于任意一个连通网的最小生成树来说，在要求总的权值最小的情况下，最直接的想法就是将连通网中的所有边按照权值大小进行升序排序，从小到大依次选择。

由于最小生成树本身是一棵生成树，所以需要时刻满足以下两点：

- 生成树中任意顶点之间有且仅有一条通路，也就是说，生成树中不能存在回路；
- 对于具有 n 个顶点的连通网，其生成树中只能有 $n - 1$ 条边，这 $n - 1$ 条边连通着 n 个顶点。

连接 n 个顶点在不产生回路的情况下，只需要 $n - 1$ 条边。

思路

所以克鲁斯卡尔算法的具体思路是：将所有边按照权值的大小进行升序排序，然后从小到大一一判断，条件为：如果这个边不会与之前选择的所有边组成回路，就可以作为最小生成树的一部分；反之，舍去。直到具有 n 个顶点的连通网筛选出来 $n - 1$ 条边为止。筛选出来的边和所有的顶点构成此连通网的最小生成树。

判断是否会产生回路的方法为：在初始状态下给每个顶点赋予不同的标记，对于遍历过程的每条边，其都有两个顶点，判断这两个顶点的标记是否一致，如果一致，说明它们本身就处在一棵树中，如果继续连接就会产生回路；如果不一致，说明它们之间还没有任何关系，可以连接。

过程

假设遍历到一条由顶点 A 和 B 构成的边，而顶点 A 和顶点 B 标记不同，此时不仅需要将顶点 A 的标记更新为顶点 B 的标记，还需要更改所有和顶点 A 标记相同的顶点的标记，全部改为顶点 B 的标记。

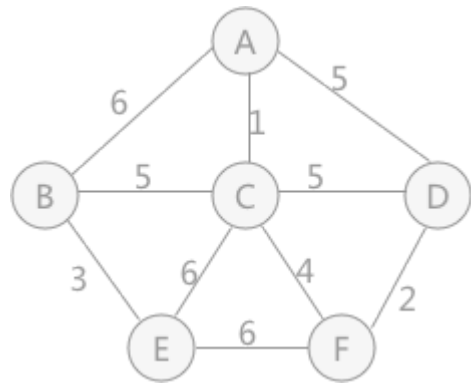
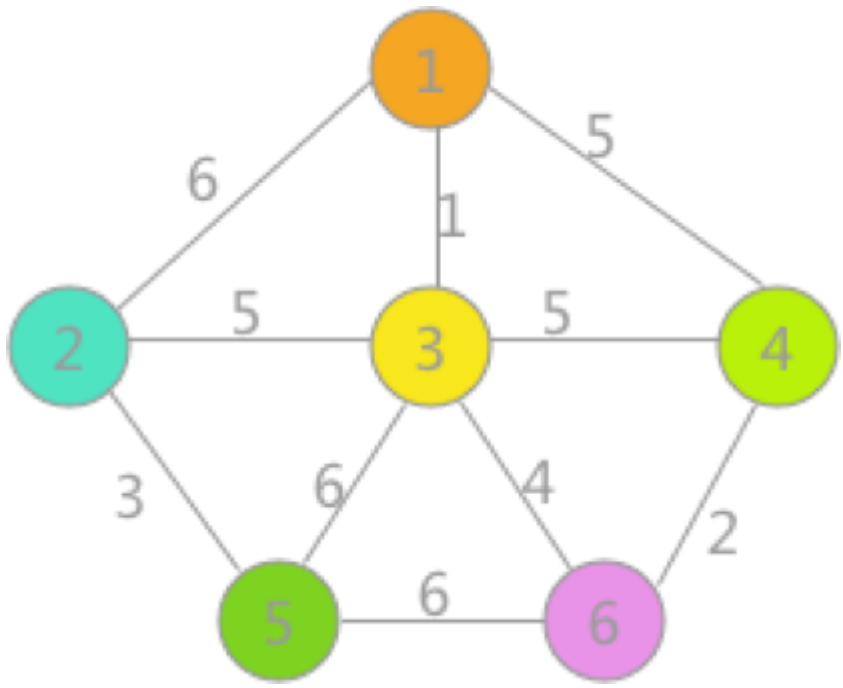


图 1 连通网

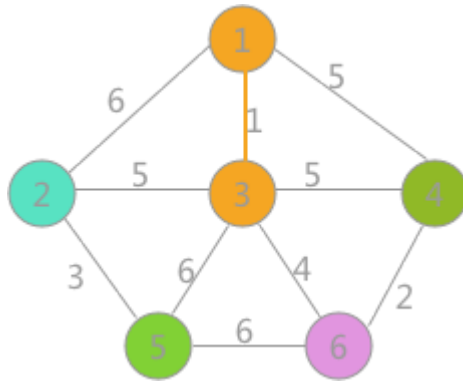
例如，使用克鲁斯卡尔算法找图 1 的最小生成树的过程为：

首先，在初始状态下，对各顶点赋予不同的标记（用颜色区别），如下图所示：



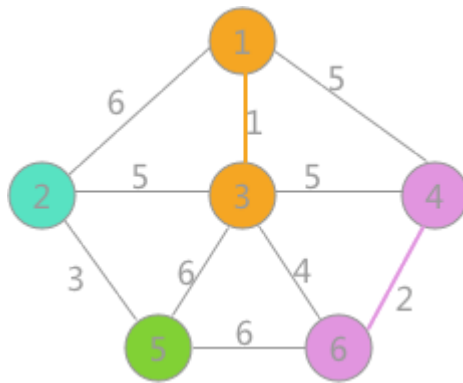
(1)

对所有边按照权值的大小进行排序，按照从小到大的顺序进行判断，首先是 (1, 3)，由于顶点 1 和顶点 3 标记不同，所以可以构成生成树的一部分，遍历所有顶点，将与顶点 3 标记相同的全部更改为顶点 1 的标记，如 (2) 所示：



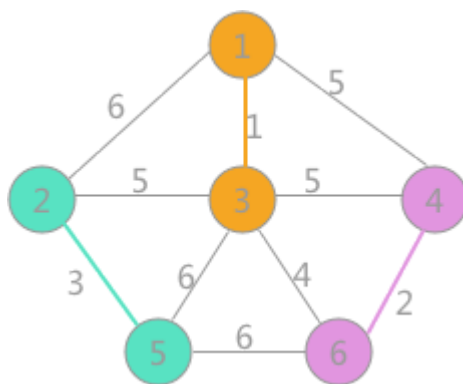
(2)

其次是 (4, 6) 边，两顶点标记不同，所以可以构成生成树的一部分，更新所有顶点的标记为：



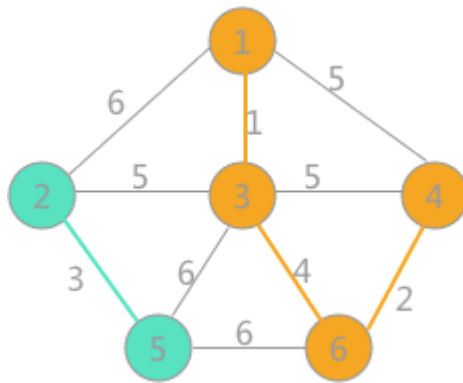
(3)

其次是 (2, 5) 边，两顶点标记不同，可以构成生成树的一部分，更新所有顶点的标记为：



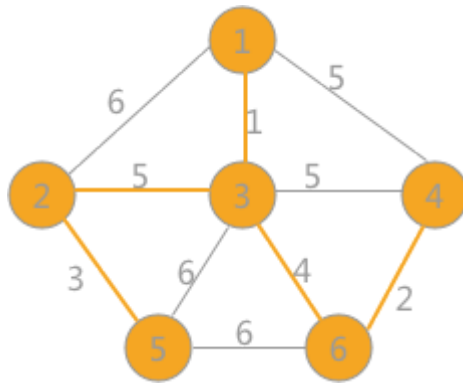
(4)

然后最小的是 3, 6 边，两者标记不同，可以连接，遍历所有顶点，将与顶点 6 标记相同的所有顶点的标记更改为顶点 1 的标记：



(5)

继续选择权值最小的边，此时会发现，权值为 5 的边有 3 个，其中 (1, 4) 和 (3, 4) 各自两顶点的标记一样，如果连接会产生回路，所以舍去，而 (2, 3) 标记不一样，可以选择，将所有与顶点 2 标记相同的顶点的标记全部改为同顶点 3 相同的标记：



(6)

当选取的边的数量相比与顶点的数量小 1 时，说明最小生成树已经生成。所以最终采用克鲁斯卡尔算法得到的最小生成树为 (6) 所示。

最小生成树

```
#include <bits/stdc++.h>
using namespace std;

const int N = 2e5 + 10;

int p[N]; //并查集数组，p[i]存储i的祖宗节点

struct Edge
```

```

{
    int u, v, w;

    bool operator<(const Edge &rhs) const
    {
        return w < rhs.w;
    }
} e[N];

int find(int x) //并查集查找x的祖宗节点
{
    if (p[x] != x) p[x] = find(p[x]);
    return p[x];
}

int main()
{
    int n, m, u, v, w, ans = 0, cnt = 0; //cnt表示已加入最小生成
    树的边的个数
    cin >> n >> m;
    for (int i = 0; i < m; i++)
    {
        cin >> u >> v >> w;
        e[i] = {u, v, w};
    }
    sort(e, e + m); //对所有边权从小到大排序

    for (int i = 1; i <= n; i++) p[i] = i; //初始化并查集数组

    for (int i = 0; i < m; i++) //从小到大枚举所有边
    {
        u = e[i].u, v = e[i].v, w = e[i].w;
        u = find(u), v = find(v); //分别查找u和v的祖宗节点
        if (u != v) //两个点不在一个集合中
        {
            p[u] = v; //合并集合
            ans += w;
            cnt++;
        }
    }
}

```

```

    }
    if (cnt < n - 1) cout << "impossible"; //边数不够，则不连通
    else cout << ans;
}

```

拓扑排序

有向无环图

定义

边有向，无环。英文名叫 Directed Acyclic Graph，缩写是 DAG。一些实际问题中的二元关系都可使用 DAG 来建模。

性质

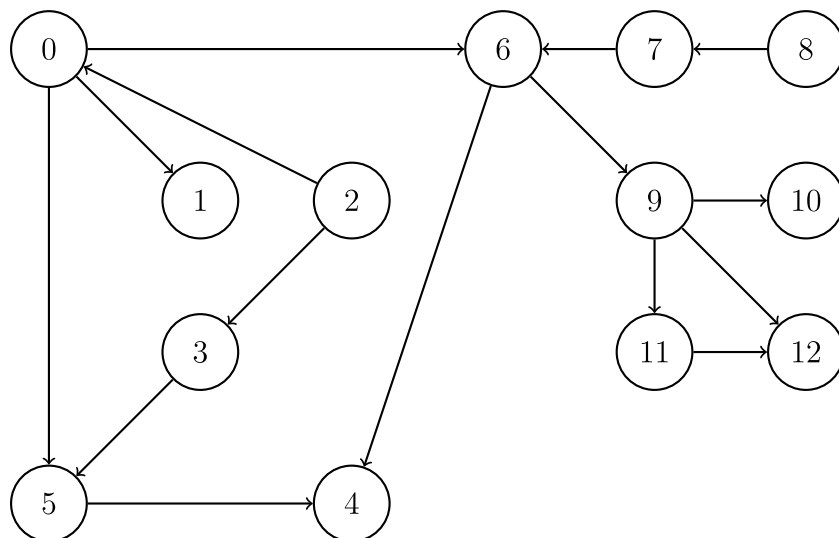
能拓扑排序的图，一定是有向无环图；

如果有环，那么环上的任意两个节点在任意序列中都不满足条件了。

有向无环图，一定能拓扑排序；

（归纳法）假设节点数不超过 k 的有向无环图都能拓扑排序，那么对于节点数等于 k 的，考虑执行拓扑排序第一步之后的情形即可。

实际上的拓扑排序是在更复杂的图上进行的，以下图为例：



对其排序的一种合法的结果就是：
 $2 \rightarrow 8 \rightarrow 0 \rightarrow 3 \rightarrow 7 \rightarrow 1 \rightarrow 5 \rightarrow 6 \rightarrow 9 \rightarrow 4 \rightarrow 11 \rightarrow 10 \rightarrow 12$ 。

但是如果图上出现了一个环，那么，我们就无法对图进行拓扑排序了。这也就是上面我们说的结论：只有有向无环图(DAG)上才可以进行拓扑排序。

因此我们可以说：**在一个 DAG（有向无环图）中，我们将图中的顶点以线性方式进行排序，使得对于任何的顶点 u 到 v 的有向边 (u, v) ，都可以有 u 在 v 的前面。**

还有给定一个 DAG，如果从 i 到 j 有边，则认为 j 依赖于 i 。如果 i 到 j 有路径（ i 可达 j ），则称 j 间接依赖于 i 。

拓扑排序的目标是将所有节点排序，使得排在前面的节点不能依赖于排在后面的节点。

Kahn 算法

基本原理

Kahn 算法基于以下思想实现：

1. 一条入边即为一条依赖，有多少入度，就有多少个依赖。
2. 对于所有未访问的节点，我们必须从入度为 0 的节点开始访问。
3. 访问一个节点后，其节点的出边所造成的依赖消失，该出边对应的入度就减少。

因此，Kahn 算法实现步骤如下：

1. 初始状态下，集合 S 装着所有入度为 0 的点， L 是一个空列表。
2. 每次从 S 中取出一个点 u （可以随便取）放入 L ，然后将 u 的所有边 $(u, v_1), (u, v_2), (u, v_3) \cdots$ 删除。对于边 (u, v) ，若将该边删除后点 v 的入度变为 0，则将 v 放入 S 中。
3. 不断重复以上过程，直到集合 S 为空。
4. 检查图中是否存在任何边或者还有节点没有被访问过，如果有，那么这个图一定至少存在一条环路，否则返回 L ， L 中顶点的顺序就是拓扑排序的结果。

拓扑排序

```
#include <bits/stdc++.h>
using namespace std;

const int N = 1e3 + 5;
int n, m, x, y;
vector<int> G[N]; // 邻接表表示图
queue<int> q;
vector<int> ans; // 存储拓扑排序的结果
int cnt[N], tpc; // 记录每个节点的入度, 以及已访问的节点数量

// 使用拓扑排序判断图中是否存在环
bool kahn() {
    // 将入度为 0 的点入队
    for (int i = 1; i <= n; i++) {
        if (!cnt[i])
            q.push(i);
    }

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        tpc++; // 统计访问过的节点数量
        ans.push_back(u); // 压入已访问节点列表
        for (int i = G[u].size() - 1; i >= 0; i--) {
            int v = G[u][i];
            cnt[v]--;
            if (!cnt[v]) // 入度为 0, 没有任何依赖, 可以访问
                q.push(v);
        }
    }

    // 如果所有节点都访问了, 说明不存在环, 否则有环
    return tpc == n;
}

int main() {
    cin >> n >> m;
```

```

// 构建图
while (m--) {
    cin >> x >> y;
    G[x].push_back(y);
    cnt[y]++; // 统计各个节点的入度
}

// 判断是否存在环
if (!kahn())
    cout << "loop exist.";
else {
    cout << "loop not exist." << endl;
    // 输出拓扑排序结果
    for (int i = 0; i < ans.size(); i++)
        cout << ans[i] << " ";
    // for(auto x : ans) cout << x << " ";
}
return 0;
}

```

蓝桥杯真题

电动车（蓝桥杯Java2023B组国赛）

如果我们对 Kruskal 算法 比较熟悉的话，是不是可以想到 Kruskal 算法 里面的对于边排序的贪心思想实际上也可以运用在本题当中。

本题要求一个至少需要多少电量，实际上是求的题目给定图的最小生成树的边权最大值最小。

而 Kruskal 算法 就是将边权进行升序排序后再一个一个的放入检查的。所以我们直接用就可以了。最后判断一下是不是选了正好 $n - 1$ 条边，也就是是否正好能找到一个完整的最小生成树，如果不行的话别忘记再输出 -1

```

#include <bits/stdc++.h>
using namespace std;

```

```

int n, m;
struct UF
{
    std::vector<int> f, siz;
    UF(int n) : f(n), siz(n, 1) { std::iota(f.begin(), f.end(),
0); }
    int find(int x)
    {
        while (x != f[x])
            x = f[x] = f[f[x]];
        return x;
    }
    bool same(int x, int y) { return find(x) == find(y); }
    bool merge(int x, int y)
    {
        x = find(x);
        y = find(y);
        if (x == y)
            return false;
        siz[x] += siz[y];
        f[y] = x;
        return true;
    }
    int size(int x) { return siz[find(x)]; }
};

int main()
{
    cin >> n >> m;
    UF uf(n);
    vector<array<int, 3>> a(m);
    for (int i = 0; i < m; ++i)
    {
        int u, v, w;
        cin >> u >> v >> w;
        u--, v--;
        a[i] = {w, u, v};
    }
    sort(a.begin(), a.end());
    int x = 0;

```

```

    for (int i = 0; i < m; ++i)
    {
        int w = a[i][0], u = a[i][1], v = a[i][2];
        if (!uf.same(u, v))
        {
            x++;
            if (x == n - 1)
            {
                cout << w << '\n';
                return 0;
            }
            uf.merge(u, v);
        }
    }
    cout << -1 << '\n';
    return 0;
}

```

发现环（蓝桥杯C/C++2017B组国赛）

本题是一个 N 个节点的树额外又加了一条边，那么显然就是有且只有一个环的。

根据树的性质与拓扑排序的算法流程，可以简单的知道只需要做一遍拓扑排序然后剩下没法进行拓扑排序的点就是环上面的点。

当然本题用 DFS 也是可以的，具体代码留给同学们自己实现。

```

#include<bits/stdc++.h>
using namespace std;
const int N=1e5+1;
int n;
int in[N];
vector<int>e[N];
bool vis[N]; //标记入度是否为1，即该点是否在环上
void topo() //拓扑排序
{
    queue<int>q;
    for(int i=1;i<=n;i++)

```

```

        if(in[i]==1)//标记入度为1的点
        {
            q.push(i);
            vis[i]=true;
        }
    while(!q.empty())
    {
        int u=q.front();
        q.pop();
        for(int i=0;i<e[u].size();i++)
        {
            int v=e[u][i];
            in[v]--;
            if(in[v]==1)//标记入度为1的点
            {
                q.push(v);
                vis[v]=true;
            }
        }
    }
}

void print()//输出未标记的点
{
    for(int i=1;i<=n;i++)
        if(!vis[i])cout<<i<<' ';
}

int main()
{
    cin>>n;
    for(int i=1;i<=n;i++)
    {
        int u,v;
        cin>>u>>v;
        e[u].push_back(v);
        e[v].push_back(u);//双向建边
        in[u]++;
        in[v]++;//两点的入度都增加
    }
    topo();
}

```

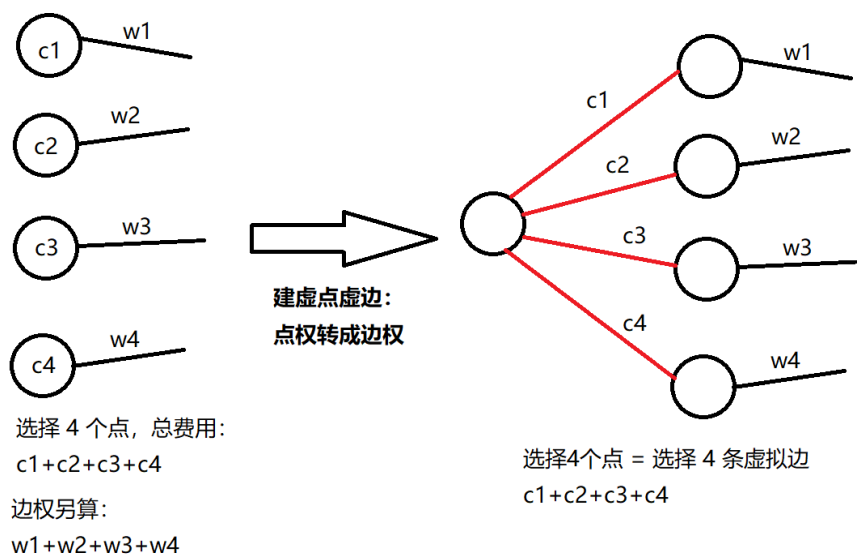
```
print();  
return 0;  
}
```

习题

集散系统

把蒙古包看作结点，供电线路看作带权边，这样就变成了图上的选边（建电线）和选点（造发电站），使得费用最小问题。

同时选边和选点比较麻烦，不好处理。于是用一个技巧，建虚拟点，把选点变成选边。



像这样，选点变成了选边，于是转换成整体都是选边，使得所有结点连在一起，并且边权之和最小的问题。这一看就知道是最小生成树问题了。

总结：多建一个虚点，编号为 0 或者 $n + 1$ ，然后把 $1 \sim n$ 结点都和虚点连边，边权就是对应的点权（建电站的费用）。然后在这个图上跑最小生成树就可以了。

参考代码：

```
#include<bits/stdc++.h>  
using namespace std;  
  
const int N = 1e3+5;
```

```

int n, v, p, cnt;
struct edge{
    int u, v, w;
    bool operator<(const edge& other)const {
        return w < other.w;
    }
}a[N*N];
int fa[N];
int find(int x){
    if(fa[x] != x) fa[x] = find(fa[x]);
    return fa[x];
}
void merge(int x, int y) {
    x = find(x); y = find(y);
    fa[x] = y;
}
int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) {
        cin >> v;
        a[++cnt] = {0, i, v}; // 虚边, 虚点为 0
    }
    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= n; j++)
        {
            cin >> p;
            if(i>j) {
                a[++cnt] = {i,j,p};
            }
        }
    sort(a+1, a+cnt+1);
    int ans = 0;
    for(int i = 0; i <= n; i++) fa[i] = i;
    for(int i = 1; i <= cnt; i++) {
        if(find(a[i].u) != find(a[i].v)) {
            ans += a[i].w;
            merge(a[i].u, a[i].v);
        }
    }
}

```



```
    }  
    cout << ans;  
    return 0;  
}
```

摇一摇广告

100 分做法：拓扑+处理环

现在图上面可能有环了。经过上面性质 C 的做法，我们可以发现，经过拓扑遍历处理后，图上就只剩下环上的结点可能会出现连续的长度达到 k 的链路。而且由于每个结点只有一条出边，因此这些环也一定是独立的，不会出现交叉形成环的情况。因此，我们只需遍历每个环，看这些环上需要删除几个结点。

需要注意，在上面拓扑处理后，环上有的结点的 $dis[] \neq 0$ 。我们就得分析一下该如何处理这些结点：

- 如果 $dis[u] = k$ ，那么意味着 u 这个结点必须删掉。
- 如果 $dis[u] < k$ ，那么意味着该结点往后 $k - dis[u]$ 个结点内必须至少有一个结点要删除。

这么去分类想可能有些复杂，且没法用上面的贪心删除的策略。考虑到每连续 $k + 1$ 个结点至少要删除一个，于是我们可以在环上枚举任意连续 $k + 1$ 个结点中的每个结点作为删除的起始点，然后去计算对应的环上的总的删除的结点数量，求出其中的最小值。最坏情况下，每个环上的结点都都循环遍历 $k + 1$ 次，假设所有结点都在环上，那么最坏情况下的时间复杂度为 $O(n \times (k + 1))$ 。也不会超时。

这部分思路也验证了上面我们说的：“很多时候，题目的部分分的设置是为了引导我们想出正解的”这句话。

参考代码：

```
#include<bits/stdc++.h>  
const int N = 3e5+5;  
int n, m, k, p[N], in[N], dis[N], ans;  
void topo() {  
    queue<int> q;  
    for(int i = 1; i <= n; i++) {  
        if(!in[i]) q.push(i);  
    }  
}
```

```

    }
    while(!q.empty()) {
        int u = q.front(); q.pop();
        in[p[u]]--;
        if(dis[u] == k) ans++; // 删除该结点, dis[p[u]] = 0
        else dis[p[u]] = max(dis[p[u]], dis[u] + 1);
        if(!in[p[u]]) q.push(p[u]);
    }
}

int calc(int u){ // 计算从 u 开始删除, 环上需要删除几个结点
    int v = p[u], res = 1, d = -1;
    while(v != u){
        d = max(d+1, dis[v]);
        if(d == k){
            res++;
            d = -1;
        }
        v = p[v];
    }
    return res;
}

int deal(int u) {
    int s = u, cnt = n+1; // cnt: 当前环上需要删除的最少结点数量
    for(int i = 1; i <= k+1; i++) {
        cnt = min(cnt, calc(s));
        s = p[s];
    }
    u = s;
    do { // 将当前结点所在的环全部标记掉
        in[u] = 0;
        u = p[u];
    } while(u != s);
    return cnt;
}

int main(){
    cin >> n >> m >> k;
    for(int i = 1, u, v; i <= m; i++) {
        cin >> u >> v;
        in[v] ++;
    }
}

```

```
        p[u] = v;
    }
    topo();
    for(int i = 1; i <= n; i++) {
        if(in[i]) ans += deal(i); // 未处理的环上结点
    }
    cout << ans;
    return 0;
}
```